Week 6: Advanced JavaScript + Async Programming

1. Callbacks, Promises

Callbacks are functions passed as arguments to other functions and executed after a task is completed. They are useful for handling asynchronous operations. However, nested callbacks can make code complex. Promises were introduced to manage asynchronous tasks in a cleaner way. A promise can have three states: pending, fulfilled, or rejected. Promises improve readability and error handling.

2. Async/Await

Async/await is a modern JavaScript feature used to simplify asynchronous programming. The `async` keyword is used before a function, and `await` pauses execution until a promise is resolved. This makes asynchronous code look more like synchronous code, improving readability and maintainability.

3. Fetch API, API Handling

The Fetch API is used to request data from servers or external APIs. It allows developers to send HTTP requests like GET, POST, PUT, and DELETE. API handling is essential for retrieving, sending, and updating data in web applications. This concept is widely used in frontend-backend communication.

4. Error Handling (try-catch)

Error handling is important for preventing application crashes. The `try-catch` block is used to catch and manage runtime errors. Developers can provide fallback solutions or error messages when something unexpected happens. This improves application stability and user experience.

Conclusion

This week focused on advanced JavaScript and asynchronous programming. By learning callbacks, promises, async/await, Fetch API, and error handling, a deeper understanding of real-world JavaScript programming was developed.